FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Franco Gerard C. Isip
Section:	AN43
Professor:	Ms. Crisola G. Tan
Date:	October 13, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

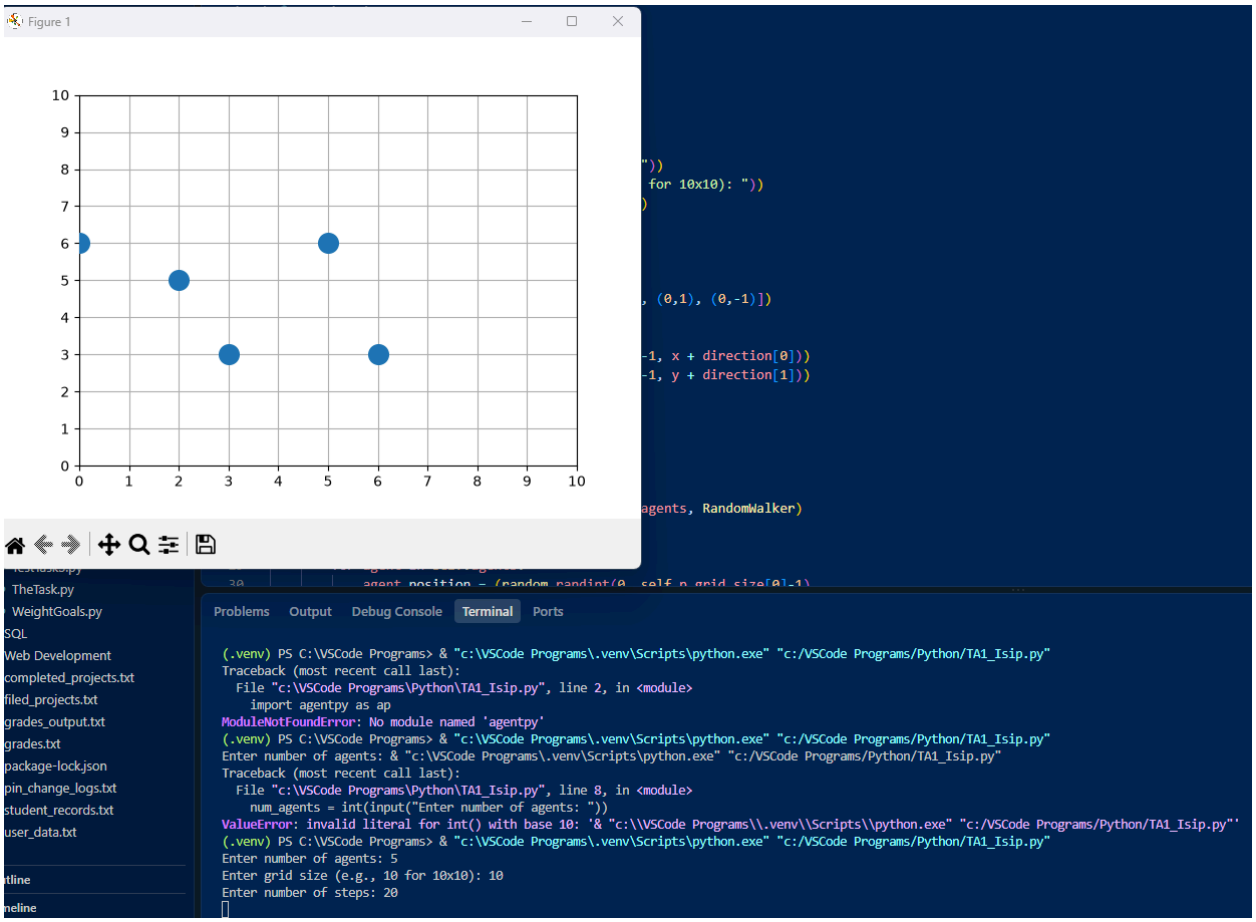
scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.



Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

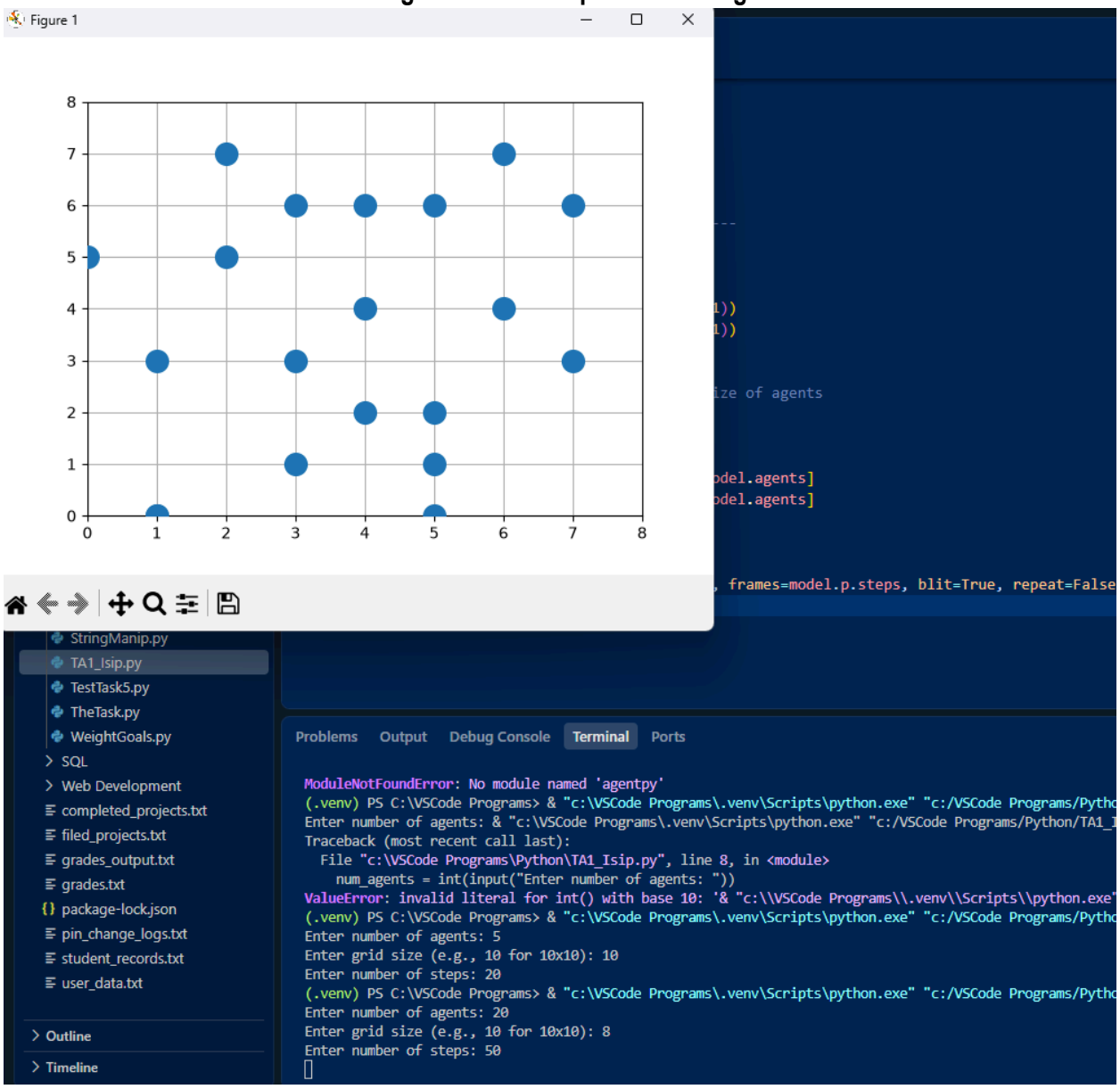
Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.



Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
from collections import Counter
import random
import statistics

import agentpy as ap
import matplotlib.animation as animation
import matplotlib.pyplot as plt


DIRECTIONS = [(1, 0), (-1, 0), (0, 1), (0, -1)]
PREFERRED_DIRECTION = (1, 0)  # East / right
PREFERRED_DIRECTION_BIAS = 0.7


def read_positive_int(prompt):
    """Read a positive integer from the console."""
    while True:
        try:
            value = int(input(prompt))
            if value > 0:
                return value
        except ValueError:
            pass
        print("Enter a whole number greater than zero.")


class RandomWalker(ap.Agent):
    def step(self):
        x, y = self.position
        width, height = self.model.p.grid_size
        possible_positions = []

        for dx, dy in DIRECTIONS:
            next_position = (x + dx, y + dy)
```

```
            if 0 <= next_position[0] < width and 0 <= next_position[1] <
height:
                possible_positions.append(next_position)

        if self.model.p.avoid_others:
            occupied_positions = {
                 other.position for other in self.model.agents if other is
not self
            }
            possible_positions = [
                position for position in possible_positions
                if position not in occupied_positions
            ]

        if not possible_positions:
            self.path.append(self.position)
            return

        preferred_position = (
            x + PREFERRED_DIRECTION[0],
            y + PREFERRED_DIRECTION[1],
        )
        if (
            self.model.p.prefer_direction
            and preferred_position in possible_positions
            and random.random() < PREFERRED_DIRECTION_BIAS
        ):
            self.position = preferred_position
        else:
            self.position = random.choice(possible_positions)

        self.path.append(self.position)


class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
```

```
        width, height = self.p.grid_size

        if self.p.avoid_others:
            positions = random.sample(
                [(x, y) for x in range(width) for y in range(height)],
                self.p.agents,
            )
        else:
            positions = [
                (random.randrange(width), random.randrange(height))
                for _ in self.agents
            ]

        for agent, position in zip(self.agents, positions):
            agent.position = position
            agent.path = [position]

        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()
        self.completed_steps += 1


def create_model(label, agents, grid_size, steps, avoid_others,
prefer_direction):
    parameters = {
        "agents": agents,
        "grid_size": (grid_size, grid_size),
        "steps": steps,
        "avoid_others": avoid_others,
        "prefer_direction": prefer_direction,
    }
    model = RandomWalkModel(parameters)
    model.setup()
```

```
    model.label = label
    model.completed_steps = 0
    return model


def report_results(model):
    final_positions = [agent.position for agent in model.agents]
    position_counts = Counter(final_positions)
    x_positions = [position[0] for position in final_positions]
    y_positions = [position[1] for position in final_positions]
    mean_eastward_displacement = statistics.mean(
        agent.position[0] - agent.path[0][0] for agent in model.agents
    )
    mean_distance_travelled = statistics.mean(
        sum(
            abs(next_x - x) + abs(next_y - y)
                    for  (x,  y),  (next_x,  next_y)  in  zip(agent.path,
agent.path[1:])
        )
        for agent in model.agents
    )
    occupied_cell_fraction = len(position_counts) / (
        model.p.grid_size[0] * model.p.grid_size[1]
    )

    print(f"\n--- {model.label}: final-position analysis ---")
    print(f"Steps completed: {model.completed_steps} / {model.p.steps}")
    print(f"Final position of each agent: {final_positions}")
    print(f"Occupied cells: {len(position_counts)} / {model.p.grid_size[0]
* model.p.grid_size[1]}")
                        print(f"Agents      per      occupied      cell:
{dict(sorted(position_counts.items()))}")
    print(
        "Mean position (x, y): "
                                    f"({statistics.mean(x_positions):.2f},
{statistics.mean(y_positions):.2f})"
    )
```

```
    print(
        "Position spread (population standard deviation, x, y): "
                                    f"({statistics.pstdev(x_positions):.2f},
{statistics.pstdev(y_positions):.2f})"
    )
     print(f"Mean eastward displacement: {mean_eastward_displacement:.2f}
cells")
    print(f"Mean distance travelled: {mean_distance_travelled:.2f} cells")
                    print(f"Fraction     of     grid     cells     occupied:
{occupied_cell_fraction:.1%}")
    return {
        "eastward_displacement": mean_eastward_displacement,
        "distance_travelled": mean_distance_travelled,
        "occupied_cell_fraction": occupied_cell_fraction,
    }


def report_comparison(results):
    print("\n--- Scenario comparison (Scenario 2 minus Scenario 1) ---")
    for metric, label in (
        ("eastward_displacement", "Mean eastward displacement"),
        ("distance_travelled", "Mean distance travelled"),
        ("occupied_cell_fraction", "Fraction of grid cells occupied"),
    ):
        difference = results[1][metric] - results[0][metric]
        print(f"{label}: {difference:+.2f}")


def main():
    print("Scenario 1: agents avoid one another and prefer moving east.")
    first_agents = read_positive_int("  Number of agents: ")
    first_grid = read_positive_int("  Grid width/height: ")
    first_steps = read_positive_int("  Number of steps: ")
    if first_agents > first_grid * first_grid:
          raise ValueError("Scenario 1 cannot have more agents than grid
cells.")
```

```
    print("\nScenario 2: unbiased random walk; agents may share cells.")
    second_agents = read_positive_int("  Number of agents: ")
    second_grid = read_positive_int("  Grid width/height: ")
    second_steps = read_positive_int("  Number of steps: ")

    models = [
        create_model(
                "Avoidance + east preference", first_agents, first_grid,
first_steps,
            avoid_others=True, prefer_direction=True,
        ),
        create_model(
                    "Random walk control", second_agents, second_grid,
second_steps,
            avoid_others=False, prefer_direction=False,
        ),
    ]

    fig, axes = plt.subplots(1, 2, figsize=(13, 6))
    artists = []
    for ax, model in zip(axes, models):
        width, height = model.p.grid_size
             colors = [plt.get_cmap("tab20")(index % 20) for index in
range(model.p.agents)]
        paths = [
            ax.plot([], [], color=color, alpha=0.6, linewidth=1)[0]
            for color in colors
        ]
        scatter = ax.scatter(
            [agent.position[0] for agent in model.agents],
            [agent.position[1] for agent in model.agents],
            s=90,
            c=colors,
            edgecolors="black",
            linewidths=0.4,
        )
        ax.set_xlim(-0.5, width - 0.5)
```

```
        ax.set_ylim(-0.5, height - 0.5)
        ax.set_xticks(range(width))
        ax.set_yticks(range(height))
        ax.set_aspect("equal")
        ax.grid(True, alpha=0.35)
                    ax.set_title(f"{model.label}\n{model.p.agents}  agents,
{width}×{height}, {model.p.steps} steps")
        ax.set_xlabel("X position")
        ax.set_ylabel("Y position")
        artists.append((model, ax, paths, scatter))

      fig.suptitle("Random-walk scenario comparison — colored lines show
agent paths")
    fig.tight_layout()
    frame_count = max(model.p.steps for model in models)

    def update(_frame):
        for model, _ax, path_lines, scatter in artists:
            if model.completed_steps < model.p.steps:
                model.step()

            for line, agent in zip(path_lines, model.agents):
                path_x = [position[0] for position in agent.path]
                path_y = [position[1] for position in agent.path]
                line.set_data(path_x, path_y)
                        scatter.set_offsets([agent.position  for  agent  in
model.agents])
        return tuple(
            artist
            for _model, _ax, path_lines, scatter in artists
            for artist in (*path_lines, scatter)
        )

    ani = animation.FuncAnimation(
              fig,  update,  frames=frame_count,  interval=350,  blit=False,
repeat=False
    )
```

```
    plt.show()

    results = [report_results(model) for model in models]
    report_comparison(results)

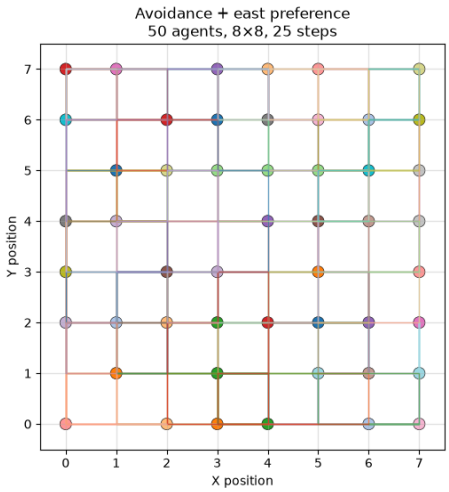

if __name__ == "__main__":
    main()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

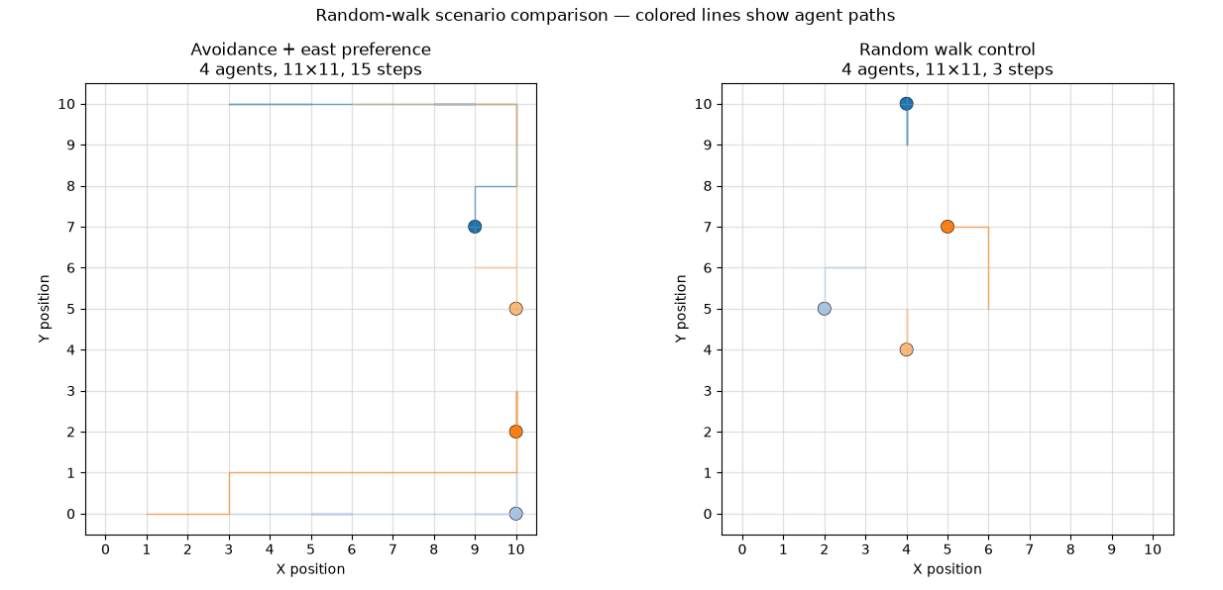
Answer the Questions ⇩

1. If the number of agents is high compared to the grid size, the grid becomes crowded. Agents that avoid one another have fewer available cells to move into and may get stuck if all neighboring cells are occupied.



For example, in this instance, 50 agents were created on an 8x8 grid, with the condition of agents avoiding each other with a preference to move eastward. Considering the total number of possible grid points (64) and the number of agents (50), there will be some agents that will most likely not be able to avoid each other and may even be positioned beside one another.

2. Increasing the number of steps gives agents more opportunities to move and generally lets them explore more of the grid. Their spread is still limited by the grid boundaries.



As observed in the image above, the variables of the number of agents and the grid size were set to be the same (4 agents on an 11x11 grid), with the steps largely varying, being 15 and 3 steps respectively. From the 15 steps, we can observe through the colored grid lines indicating the distance travelled by each agent — being very long and in accordance with its preset conditions of avoidance and eastward preference. While for the instance that only has 3 steps, while random in nature, it shows the large difference in the agents' traversal versus the 15 steps, only showing short colored grid lines.

3. Agents do not move outside the grid because the simulation only allows moves to positions within the grid's bounds.
4. The simulation can illustrate diffusion through random movement or approximate pedestrian movement using rules like avoiding others and preferring a direction. More realistic models could include obstacles, exits, different speeds, or destinations.